

# RoboPrompt: Intuitive Robot Policy Steering with Sparse Human Input

**Abstract**—End-to-end robot policies trained through imitation learning remain constrained by limited data diversity, making reliable zero-shot deployment in real-world settings challenging. Shared-autonomy methods enable human correction through teleoperation, but specialized hardware and operator training hinder deployment at scale. Other approaches incorporate human guidance as additional policy inputs, often requiring architectural changes and dedicated training for steerability, which limits their applicability across policies. We present RoboPrompt, a general-purpose, lightweight robot policy steering system that enables users to guide policy behavior through intuitive, sparse inputs, including drawn traces, target points, and coarse directional instructions. RoboPrompt decouples human-intention translation from the underlying policy: a reusable module converts human guidance into action drafts, which are refined through the diffusion or flow-matching dynamics of the base policy. By controlling action generation in noise space, RoboPrompt balances human intent with the policy prior without modifying the base policy architecture or fine-tuning it for steerability. Experiments demonstrate effective steering across Diffusion Policy,  $\pi_{0.5}$ , and FastWAM. We further use steered rollouts for online policy improvement through DAgger. After 2-3 rounds of iteration, average success rates increase by 15.5% for  $\pi_{0.5}$  across three tasks and by 21.3% across three policies (Diffusion Policy,  $\pi_{0.5}$ , FastWAM) on the Insert Bread task, while average human intervention counts decrease by 44.0% (2.86 to 1.60) and 81.9% (2.60 to 0.47), respectively.

**Index Terms**—Robot Policy Steerability, Robot Manipulation, Imitation Learning.

## I. INTRODUCTION

RECENT advances in vision-language-action (VLA) models [1]–[4] and world-action-models (WAM) [5]–[8] have significantly improved the generalization capability of robotic manipulation.

Scaling beyond laboratory settings remains constrained by scarce real-world interaction data [9], unlike the web-scale corpora available to language and vision models [10], [11]. This creates a chicken-and-egg dilemma: limited data diversity causes deployment failures, which in turn restrict deployment and further data collection.

**Human-in-the-loop deployment** addresses this dilemma by letting robots act autonomously while humans monitor and correct execution. However, end-to-end policies [3], [5], [12]–[14] offer limited intervention interfaces. Language specifies task objectives but often provides weak fine-grained control [15], [16], especially when the intended behavior is semantically ambiguous [17], [18].

Shared-autonomy approaches allow humans to correct failures through teleoperation [13], [19], [20], but require

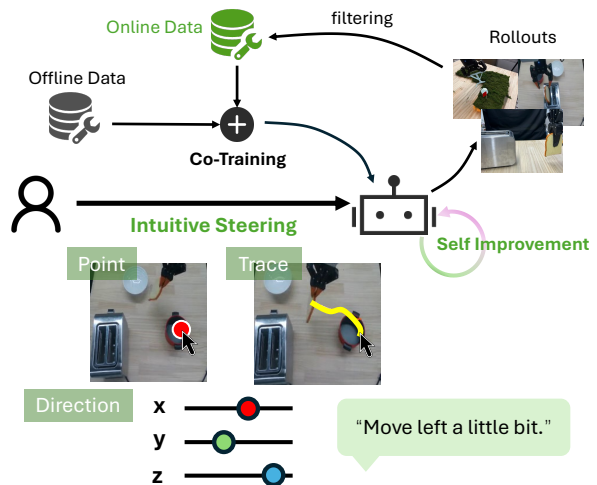


Fig. 1: **Overview of RoboPrompt.** RoboPrompt aims to enable human-in-the-loop deployment of robot policies, collecting failure-recovery rollout data that can drive continual policy improvement. It supports intuitive multimodal steering through inputs, including points, traces, and coarse directional instructions. These human-guided rollouts are collected and combined with offline data for co-training, forming a closed-loop framework for task completion and self-improvement without teleoperation.

additional hardware and operator training, hindering adoption by everyday users after large scale deployment.

We argue that **scalable human-in-the-loop deployment requires an intuitive steering interface with minimal physical and cognitive burden**. Drawing 2D sketches on camera observations or providing coarse directional instructions could enable task completion while collecting corrective data for policy improvement.

Realizing such an interface presents several challenges. First, unlike teleoperation, natural human instructions only provide partial information rather than full 6D actions with temporal dynamics. Second, long-horizon prompt following is non-trivial under action-chunked policies. Effective correction may require actions outside the base policy’s learned action manifold, which the policy cannot reliably produce on its own, so steering should initially prioritize human intention and gradually transition toward the policy prior across subsequent action chunks. Finally, existing prompt-conditioned approaches often rely on whole policy fine-tuning or architectural modifications [17], [21], [22], which ties steerability to a particular policy design and makes it difficult to reuse the same steering interface across pretrained policies with different architectures or weights.

We propose **RoboPrompt**, a two-stage system that trans-

lates sparse human input into executable actions, integrates them with pretrained policies, and uses steered rollouts for policy improvement. Our main contributions are:

- **From intention to action draft.** We introduce a lightweight human-intention translator based on a 0.77B VLA model [23], together with a data processing pipeline for its training. The translator converts 2D traces, points, and coarse directional instructions into spatially and temporally complete action chunks, serving as a task-agnostic yet physically plausible action generator.
- **From action draft to fine-grained movement.** We propose *Progressive Biased Denoising (PBD)*, which refines action drafts by exploiting the properties of diffusion- and flow-matching-based policies. Increasing the denoising level across action chunks extends a prompt’s influence over long-horizon behavior while smoothly shifting control toward the policy prior.
- **Steering and improving real-world policies.** RoboPrompt achieves  $\geq 95\%$  prompt alignment on Diffusion Policy [14],  $\pi_{0.5}$  [13], and FastWAM [5] without fine-tuning for steerability. We introduce Time-weighted Optimal Transport (TOT) to filter steered rollouts for DAgger training, improving task progress across real-world tasks while reducing human intervention.

## II. RELATED WORK

**Robot learning for manipulation tasks.** Learning-based visuomotor policies, ranging from task-specific robot policies [14] to large-scale robot foundation models including VLAs [12], [13] and WAMs [5], [7], have achieved increasingly strong capabilities in robotic manipulation through imitation learning. These policies are typically trained end to end to map visual observations and task instructions directly to robot actions, allowing them to benefit from large-scale robot datasets. However, this end-to-end formulation provides few interfaces for humans to intervene during policy execution.

**Policy steering.** To improve manipulation performance, recent works have explored steering a base policy with additional conditioning signals during execution. Steerable Policies [21] enable users to provide diverse prompts for refining policy behavior by translating them into language inputs to condition the policy.  $\pi_{0.7}$  [12] takes a step further toward more grounded conditioning by prompting a world model to convert subtask-level language prompts into goal images. While language is effective for specifying high-level tasks and semantic intent, language alone often cannot express the full set of fine-grained spatial and motion-level information required for effective steering [17], [24]. Inference-Time Policy Steering [25] enables training-free intervention by using visual guidance objectives to bias the sampling process of diffusion policy [14]. However, reweighting remains constrained by the base policy’s own action distribution, limiting its ability to realize corrective actions outside the policy’s learned action manifold that may be

necessary for effective failure recovery. Instead, RoboPrompt uses an independent module to convert human input into an action draft and refines it using Progressive Biased Denoising (PBD) and Noise-Augmented Flow Reversal Steering (NAFRS) with a tunable initial denoising step, preserving human intent while adjusting its influence relative to the base policy prior.

**Interactive robot learning.** Interactive robot learning enables a learned policy to receive human guidance or intervention during execution. Such interactions improve task completion when autonomous execution is unreliable, and the resulting trajectories can further serve as online data for policy improvement methods such as HG-Dagger [26]. Existing collaborative-control systems [20], [27], [28] commonly realize this interaction through teleoperation, where users directly provide low-level motion commands. Shared-autonomy methods such as HAJL [29] reduce the control burden by employing a diffusion-based assistive policy that refines teleoperation commands during the diffusion process, continuously interpolating between manual and autonomous control. Nevertheless, these approaches still depend on teleoperation devices and expert operators to produce precise low-level commands. To reduce this dependence on skilled teleoperation, RoboPrompt uses sparse, intuitive human guidance and relies on the base policy as an action prior to infer executable low-level behavior.

## III. SYSTEM ARCHITECTURE

Human intentions often underspecify robot actions. We seek a general mapping from sparse human guidance to action drafts that downstream policies can refine using their task-aware priors. RoboPrompt implements this mapping in two phases. **Phase I** uses a lightweight steer model to translate points, traces, and coarse directional instructions into continuous action drafts. **Phase II** refines these drafts through the downstream policy’s denoising process, balancing human intent with the policy prior without policy fine-tuning or architectural changes. We detail these two phases in the following subsections.

### A. Phase I: Action Drafting

Intuitively, humans often teach others how to perform a task by drawing a sketch, pointing to a desired location, or giving a coarse directional instruction. RoboPrompt aims to provide robots with the same lightweight forms of guidance. We therefore define three prompt types: point and trajectory prompts are given as visual inputs directly overlaid on the camera observation, while global action prompts are given as text inputs appended to the task instruction. Though VLMs can produce similar prompt labels, we find that they can be unreliable when the task contains semantic ambiguity, visual occlusion, or fine-grained manipulation that requires precise motion reasoning. We therefore implement prompt label generation as follows. For each trajectory, we construct the prompt that would have guided the robot along its actually executed future motion, as illustrated in Fig. 3.

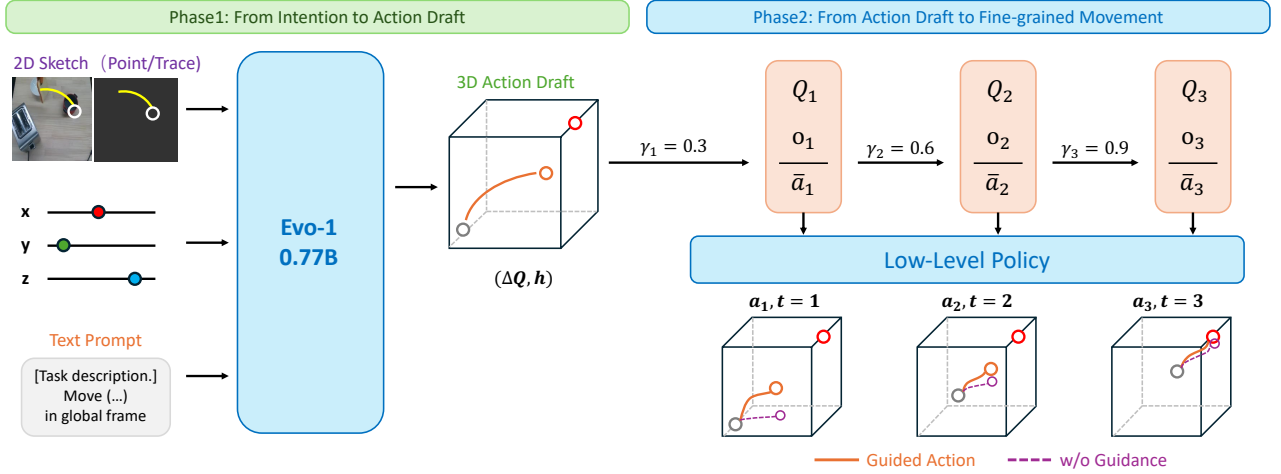


Fig. 2: **System Architecture.** RoboPrompt incorporates human intention into policy inference through two phases. In Phase I, prompt inputs are processed by a lightweight VLA to produce a temporally continuous 6D action chunk as an action draft. In Phase II, the base policy refines the action draft generated by Phase I. The fraction of noise-adding steps,  $\gamma$ , controls the balance between human guidance and the base policy.  $\gamma$  increases with successive replanning steps, gradually shifting control toward the base policy.

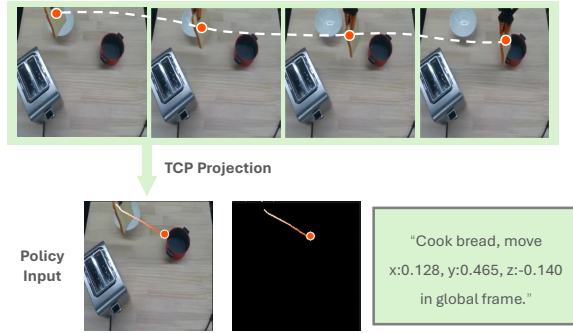


Fig. 3: **Prompt label generation.** Over the next  $n$  frames, connected TCP projections and the final TCP projection provide ground truth for trajectory and point prompts; summed and normalized actions supervise the global action prompt.

Formally, let an episode be  $\tau = \{(O_t, Q_t, a_t)\}_{t=0}^{T-1}$ , where  $O_t$  is the visual observation,  $Q_t$  is the robot TCP pose, and  $a_t \in \mathbb{R}^{d_a}$  is the low-level action. Denote the action horizon as  $H$ . We use an image projection function

$$u_t = f_{\text{proj}}(Q_t) \in \mathbb{R}^2,$$

to project the TCP onto the image plane of the robot’s fixed environment camera.

**Point prompt.** A point prompt specifies where the TCP should arrive after the short-horizon motion. For a sample starting at time  $t$ , the hindsight target is the projected TCP endpoint

$$p_t^{\text{pt}} = u_{\bar{t}}, \quad \bar{t} = \min(t + H - 1, T - 1).$$

The model input is an image with  $p_t^{\text{pt}}$  drawn on top of  $O_t$ ,

$$\tilde{O}_t^{\text{pt}} = \text{Overlay}_{\text{pt}}(O_t, p_t^{\text{pt}}).$$

**Trajectory prompt.** A trajectory prompt gives a sparse 2D trace of the desired TCP motion over the action horizon. Its ground truth is the projected future TCP sequence

$$P_t^{\text{traj}} = (u_t, u_{t+1}, \dots, u_{\bar{t}}), \quad \bar{t} = \min(t + H - 1, T - 1),$$

and the visual input is produced by connecting these sparse points into a line on the observation,

$$\tilde{O}_t^{\text{traj}} = \text{Overlay}_{\text{traj}}(O_t, P_t^{\text{traj}}).$$

**Global action prompt.** A global action prompt describes the coarse Cartesian displacement that the robot should make in the base frame. Its hindsight target is obtained by accumulating the raw XYZ actions over the same horizon,

$$g_t = \text{Norm}_g \left( \sum_{k=0}^{H-1} a_{t+k}^{xyz} \right) \in [-1, 1]^3.$$

It is then converted into a textual prompt, as shown in Fig. 3. For all three prompts, we normalize the input and apply random position and scale perturbations during training, considering that humans always provide imprecise prompts.

Phase I adopts a lightweight 0.77B-parameter VLA, Evo-1 [23], as the action-draft policy architecture, as shown in Fig. 2. For point and trajectory prompts, the prompt is rendered directly on the environment-camera image and used as the primary visual input. To make the steering signal explicit and prevent it from being diluted by scene appearance, we also render a prompt-only image without the camera background and feed it as the second image input. For global action prompts, the normalized Cartesian displacement is verbalized and appended to the original task description as part of the text input. The model predicts a temporally continuous action chunk,

$$a_I = \pi_{\phi}(O_t^{(1)}, O_t^{(2)}, q_t),$$

where  $O_t^{(1)}$  is the prompted camera observation,  $O_t^{(2)}$  is the prompt-only image for visual prompts, and  $q_t$  is the task instruction augmented with the global action prompt when applicable.

In practice, we train the Phase I steering model on 500 play-data trajectories collected on the same robot platform used in our experiments, with prompt labels generated using

the procedure described above. Auxiliary losses are applied to supervise the training process as follows:

$$\mathcal{L}_I = \mathcal{L}_{\text{act}}(a_I, a_{t:t+H-1}) + \lambda_{\text{aux}} \mathcal{L}_{\text{aux}}.$$

The auxiliary objective combines supervision for the three prompt types,

$$\mathcal{L}_{\text{aux}} = \mathcal{L}_{\text{pt}} + \mathcal{L}_{\text{traj}} + \mathcal{L}_{\text{global}},$$

The point loss  $\mathcal{L}_{\text{pt}}$  and trajectory loss  $\mathcal{L}_{\text{traj}}$  supervise the target point and trajectory at the pixel level in the visual input space. For global action prompts, we use  $\mathcal{L}_{\text{global}} = \text{MSE}(\hat{g}_t, g_t)$ , where  $\hat{g}_t$  is the predicted global action and  $g_t$  is the normalized ground-truth displacement defined above. At this stage, the model produces plausible actions consistent with human intent as well as current observation. The resulting action draft is further refined in Phase II.

### B. Phase II: From Action Draft to Fine-grained Movement

Phase II aims to refine the action draft produced by Phase I with the base policy prior. Current mainstream robot policies commonly generate actions through diffusion or flow-matching processes. Following [29], we leverage the mathematical structure of iterative denoising to fuse human intention with the policy output. To enable long-horizon prompts to influence policy behavior across action chunks while smoothly transitioning control from human intention toward the policy prior, we propose Progressive Biased Denoising (PBD). Let  $m$  denote the replanning chunk index after a prompt is issued,  $N$  be the total denoising steps of the base policy, and  $k^m$  be the current denoising step, updated by a fixed increment  $\Delta k$ :

$$k^m = \min(k^0 + m\Delta k, N), \quad \gamma^m = k^m/N.$$

Here  $\gamma^m$  is the chunk-wise normalized denoising level used for refinement. In the basic PBD formulation, we perturb the Phase I draft at each chunk with the progressively increasing  $\gamma^m$  and denoise it with the base policy:

$$z_{\gamma^m} = (1 - \gamma^m)a_I + \gamma^m\epsilon, \quad \epsilon \sim \mathcal{N}(0, I),$$

$$a_{II} = \mathcal{D}_\theta(z_{\gamma^m}, \gamma^m \rightarrow 0),$$

where  $z_{\gamma^m}$  denotes the noisy action at denoising level  $\gamma^m$ , and  $\mathcal{D}_\theta$  denotes the denoising process of the downstream policy. Note that  $\gamma$  is a tunable control ratio that regulates the influence of human guidance relative to the policy's original prediction. When  $\gamma = 0$ , the Phase II output is exactly the Phase I draft, while when  $\gamma = 1$ , the Phase II output falls back to the base policy action without any human guidance.

A single human sketch or directional prompt often corresponds to motion longer than one action chunk. The monotonic schedule of  $\gamma^m$  preserves the same Phase I human input across multiple replanning chunks: the first chunk follows the human input strongly, while later chunks gradually reintroduce the base policy prior.

However, the action change induced by denoising is not uniform across denoising time: even a small number of

denoising steps can produce strong corrections, as further illustrated in our denoising-dynamics experiment in Sec. 3. This is undesirable for cross-action-chunk prompting, where control should transition smoothly rather than abruptly from human intention to the policy prior. To avoid the strong correction caused by directly adding random noise to  $a_I$ , we build on Flow Reversal Steering [30] and introduce Noise-Augmented Flow Reversal Steering (NA-FRS). NA-FRS first runs the learned denoising dynamics backward from the clean Phase I action to the target denoising level as FRS:

$$z_{\gamma^m}^{\text{rev}} = \mathcal{R}_\theta(a_I, 0 \rightarrow \gamma^m),$$

where  $\mathcal{R}_\theta$  is the reverse integration of the base policy denoising dynamics. We write the implied reverse noise as

$$\epsilon_{\text{rev}} = (z_{\gamma^m}^{\text{rev}} - (1 - \gamma^m)a_I)/\gamma^m.$$

This reverse process largely preserves Phase I information while limiting the original policy performance. So we gradually corrupt the Phase I distribution as  $\gamma^m$  grows. NA-FRS therefore mixes the reverse path with pure Gaussian noise:

$$\epsilon_{\text{mix}} = \sqrt{1 - \rho^2}\epsilon_{\text{rev}} + \rho\epsilon,$$

$$z_{\gamma^m}^{\text{NA}} = (1 - \gamma^m)a_I + \gamma^m\epsilon_{\text{mix}},$$

where  $\rho \in [0, 1]$  controls the random-noise ratio. The refined action is then obtained by denoising  $z_{\gamma^m}^{\text{NA}}$  back to  $t = 0$ .

### C. Improve Policy Performance with Steered Rollouts

Successful steered rollouts may still contain oscillations, repeated gripper actions, or long suboptimal paths. Such data can pair similar observations with inconsistent or delayed actions, degrading Markovian imitation-learning policies during DAgger-style [31] training. We introduce Time-weighted Optimal Transport (TOT) to retain successful rollouts whose behavior and duration are close to expert demonstrations.

Concretely, for a steered rollout  $\tau^r = \{x_i^r\}_{i=1}^{T_r}$  and an expert trajectory  $\tau^e = \{x_j^e\}_{j=1}^{T_e}$ , we extract policy embeddings  $\phi(x)$  and define a timestep-level cost

$$C_{ij} = 1 - \frac{\phi(x_i^r)^\top \phi(x_j^e)}{\|\phi(x_i^r)\|_2 \|\phi(x_j^e)\|_2}.$$

We then compute a TOT score by matching the rollout to its closest expert trajectory while penalizing unnecessarily long executions. Let  $\bar{T}_E = \text{median}_{\tau \in \mathcal{D}_E} |\tau|$ :

$$D_{\text{OT}}(\tau^r, \tau^e) = \min_{\Pi \in \mathcal{U}(T_r, T_e)} \sum_{i=1}^{T_r} \sum_{j=1}^{T_e} \Pi_{ij} C_{ij}.$$

$$S(\tau^r) = \min_{\tau^e \in \mathcal{D}_E} [D_{\text{OT}}(\tau^r, \tau^e) + \lambda \max(0, T_r/\bar{T}_E - 1)].$$

where  $D_{\text{OT}}$  denotes the optimal-transport distance between a rollout and an expert trajectory,  $\Pi$  is the transport plan between rollout and expert timesteps, and  $\mathcal{U}(T_r, T_e)$  denotes the set of plans with uniform temporal marginals. Lower scores indicate rollouts that are both semantically close to expert behavior and not excessively long. We only keep rollouts with  $S(\tau^r)$  below a threshold for online training.

TABLE I: Steering performance of RoboPrompt across different policies and tasks. The left block compares task progress with and without steering; the right block reports steering quality (the lower the steering counts, the less human intervention is needed; the higher the prompt alignment, the more faithfully the steered behavior follows the human prompt).

| Success Rate Improvement |       |                           |                        | Steering Quality     |                   |                   |                           |
|--------------------------|-------|---------------------------|------------------------|----------------------|-------------------|-------------------|---------------------------|
| Policy                   | Task  | Progress without Steering | Progress with Steering | Avg. Steering Counts | Avg. visual input | Avg. global input | Avg. Prompt Alignment (%) |
| $\pi_{0.5}$              | Left  |                           | 77.5%                  | 2.50                 | 1.45              | 1.90              | 99.44%                    |
|                          | Right | 59.6%                     | 72.4%                  | 2.95                 | 2.50              | 2.50              |                           |
|                          | Pot   |                           | 89.8%                  | 2.35                 | 1.80              | 1.25              |                           |
| DP                       | Left  |                           | 71.3%                  | 4.80                 | 3.10              | 3.40              | 97.1%                     |
|                          | Right | 46.5%                     | 64.75%                 | 3.45                 | 1.85              | 1.95              |                           |
|                          | Pot   |                           | 57.95%                 | 4.40                 | 3.95              | 1.85              |                           |
| FastWAM                  | Left  |                           | 57.4%                  | 3.15                 | 2.15              | 2.30              | 100.0%                    |
|                          | Right | 46.2%                     | 62.3%                  | 3.10                 | 1.90              | 2.10              |                           |
|                          | Pot   |                           | 52.3%                  | 2.65                 | 1.70              | 1.55              |                           |

Following [32], the filtered online rollouts are mixed with the original offline data at a 50%/50% ratio for the next DAgger training round, gradually moving the original policy distribution toward the corrected deployment distribution while retaining the base policy prior.

#### IV. EXPERIMENT

We use a Flexiv Rizon4 robot equipped with a Robotiq 2F-85 gripper as our robot platform. For each task, we collect 100 demonstrations through teleoperation with a Sigma.7 device to train the base policy.

*Q1: Can RoboPrompt Effectively Steer Policies toward Different Task Objectives?*

We conduct steering experiments on Diffusion Policy (DP),  $\pi_{0.5}$ , and FastWAM. For each policy, we train on data containing three different task objectives, resulting in a multimodal policy distribution. RoboPrompt is then used at inference time to specify the desired task objective, as illustrated in Fig. 4.

Without steering, the learned multimodal policies exhibit different preferences over the three valid task objectives, as shown by the pie charts in Fig. 4. Note that the percentages for a given policy do not sum to 100% because some trials failed before reaching any target during inference; those failures are excluded from the task-preference distribution.

We then apply RoboPrompt to steer each policy toward each specific target. We define prompt alignment as the percentage of human interventions for which the resulting robot motion follows the direction specified by the prompt along the dimensions indicated by that input. Dimensions not specified by the prompt are excluded from this assessment. For each policy, we compute this percentage over all interventions across the three task objectives. The results are reported in Table I, which is divided into two blocks. The left block compares the task progress with and without steering: across all three policies, RoboPrompt consistently improves the success rate over the unsteered baseline. The right block evaluates the steering quality: prompt alignment

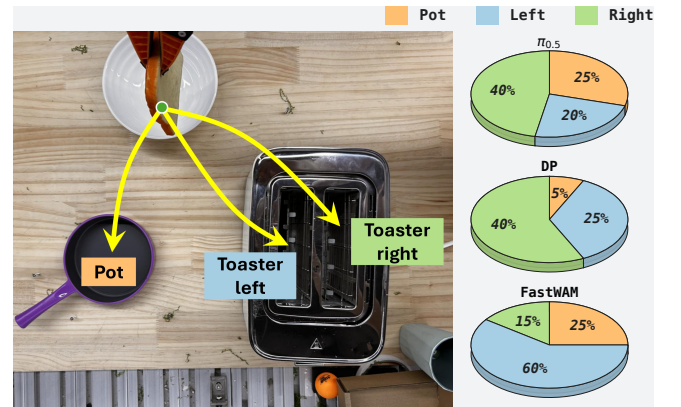


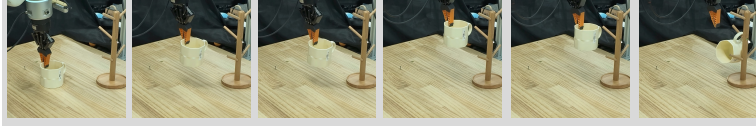
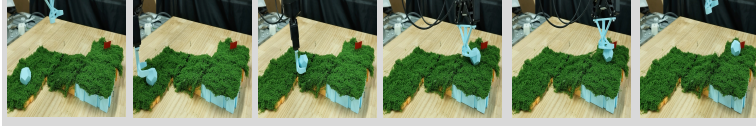
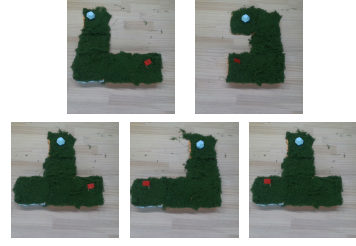
Fig. 4: **Steering experiment setup and natural policy distribution.** In the steering experiment, the robot picks up bread from the bowl and places it into one of three targets (pot, left toaster, right toaster). The training set contains 46 demonstrations per target, all using the same text description across the three targets. The right panel shows the natural distribution of the three base policies over the three targets before steering.

remains above 95% across all three policies, confirming that the steered actions follow the human prompt.

*Q2: Can RoboPrompt Help Policy Improvement via DAgger?*

Based on the filtered steered-rollout pipeline described above, we evaluate whether RoboPrompt can improve the underlying policy through DAgger. We first evaluate this with  $\pi_{0.5}$  on three real-world tasks: Insert Bread, Hang Cup, and Push Ball (Fig. 5). Each task uses 100 training demonstrations: Insert Bread uses only right-slot insertion demonstrations, unlike the insertion task in the steering experiment, and Push Ball uses 20 demonstrations for each of five different routes. At inference, Push Ball uses an unseen layout to evaluate adaptation through steering.

As shown in Fig. 6, the improvement is significant across different tasks. Each DAgger round increases the task success rate while reducing the steering counts required in the next round, indicating that a policy with moderate initial performance and frequent human intervention can gradually transition toward fully autonomous execution within only a few DAgger iterations.

**Insert Bread****Hang Cup****Push Ball****Push Ball Training Set****Eval Case**

(a)

(b)

Fig. 5: **DAGger experiment setup.** (a) *Insert Bread*: pick up bread from a bowl and insert it into the toaster’s right slot; *Hang Cup*: hang a cup by its handle on the second-highest peg; *Push Ball*: push a polyhedral ball across an uneven platform until it touches the flag. (b) Push Ball training dataset contains 5 different layouts, and is evaluated on another new layout. Steered rollouts enable task completion in the new layout and provide data improve the policy over successive DAGger rounds.

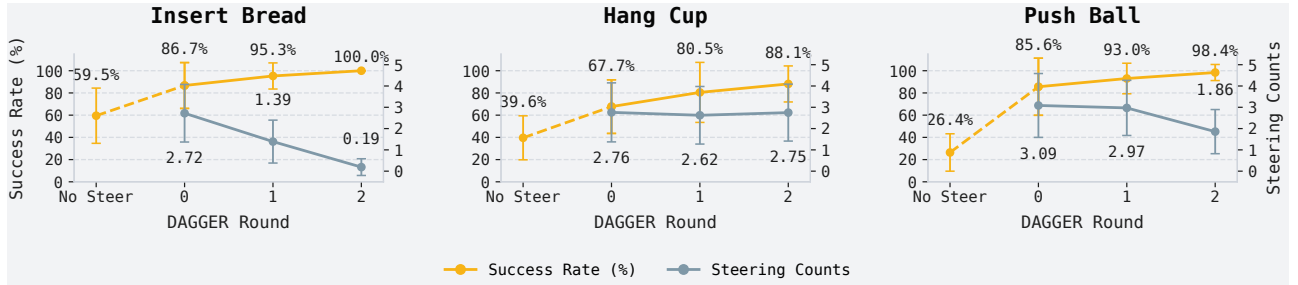


Fig. 6: Main DAGger results with  $\pi_{0.5}$  across three real-world tasks. Round 0 denotes the original policy before DAGger fine-tuning. Across the Insert Bread, Hang Cup, and Push Ball tasks, iterative training with RoboPrompt-guided corrections improves success rate, while the required steering counts are reduced or remain comparable.

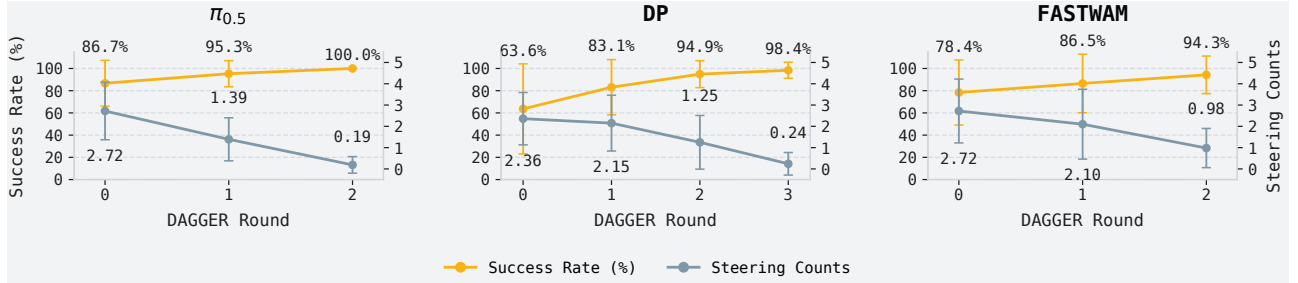


Fig. 7: DAGger performance after replacing the low-level policy. Round 0 denotes the original base policy. With additional DAGger iterations, task progress increases while the average steering counts decrease across DP,  $\pi_{0.5}$ , and FastWAM.

We next evaluate DP and FastWAM with the same DAGger protocol on the bread task and observe the same trend, as shown in Fig. 7: additional DAGger rounds improve task progress while reducing the amount of steering required during deployment. Together, these results suggest that RoboPrompt makes it easier for users to correct policy behavior through sparse, intuitive guidance while retaining the policy improvement ability of DAGger.

### Q3: How Do Denoising Dynamics Affect RoboPrompt Steering?

We study the denoising dynamics of RoboPrompt in Fig. 8. For both flow-matching and diffusion-based policies, action changes are not uniformly distributed across denoising steps. A small number of denoising steps can already move the action substantially, while the final steps at larger denoising levels are close to convergence and produce only minor changes. This explains why naively increasing the denoising level does not provide a linear control knob between the Phase I action and the base policy output.

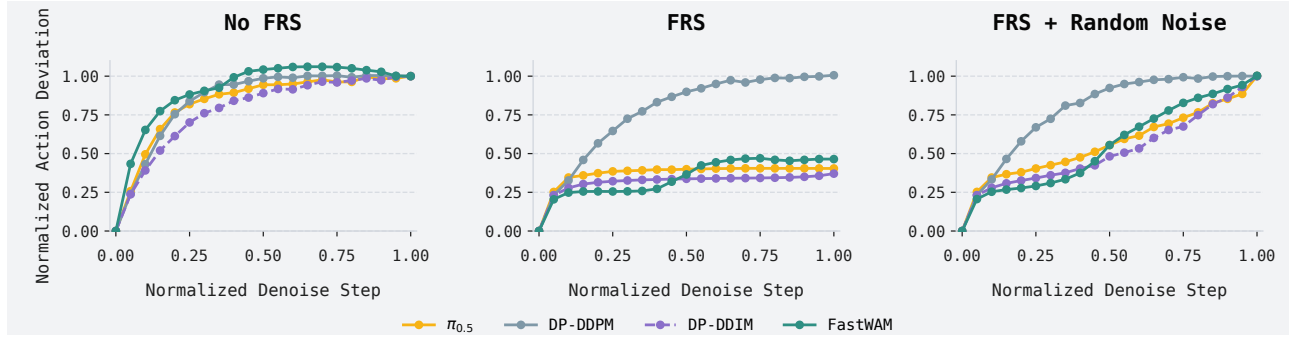


Fig. 8: **Denoising dynamics ablation.** Original flow-matching and diffusion-based policies exhibit non-uniform action changes over denoising time. FRS keeps the output closer to the Phase I action, while NA-FRS produces a smoother, approximately linear transition across denoising levels.

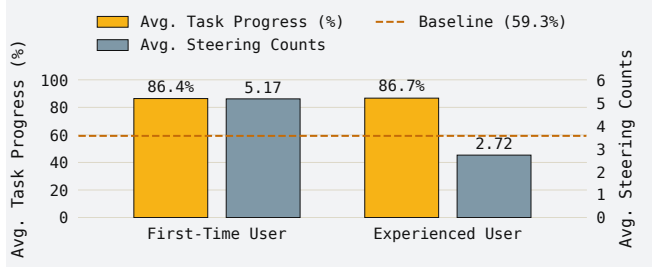


Fig. 9: Task progress and steering counts for first-time and experienced users on *bread insertion*, using the same policy checkpoint and environment. In baseline, the policy executes autonomously without human steering.

NA-FRS changes this behavior by reversing the policy dynamics from the Phase I action before refinement and then injecting controlled random noise. The reverse component keeps the refined action close to the Phase I draft, while the noise component lets the trajectory recover the behavior of pure-noise denoising as the denoising level grows. Together, they produce a smoother, near-linear action transition across denoising levels.

*Q4: Can first-time users intuitively steer robot policies with RoboPrompt?*

To assess the ease of use of RoboPrompt for first-time users, we recruit 6 volunteers with no prior experience with our system. Following a brief 5-minute introduction, each participant performs 5 trials of the *bread insertion* task using the  $\pi_{0.5}$  Round-0 toaster policy shown in Fig. 6. We compare first-time users’ average task progress and average steering counts with those of experienced users.

As shown in Fig. 9, first-time users achieve task progress comparable to experienced users and substantially above the unsteered baseline, suggesting that RoboPrompt enables new users to intuitively steer robot policies. Experienced users need fewer interventions, as familiarity with the policy helps them anticipate and prevent failures instead of recovering from them.

*Q5: How does rollout data quality affect policy improvement?*

We compare DAgger training on 20 successful trajectories selected either randomly (unfiltered) or using Time-weighted

Optimal Transport (TOT; filtered). Both settings use one training round and are evaluated across 3 tasks.

TOT-filtered data yields higher task progress across all 3 tasks (Fig. 10). Unfiltered data increases motion jitter, potentially causing failures or additional interventions; successful execution alone therefore does not ensure training quality. TOT retains smoother trajectories consistent with human intent, improving policy performance and reducing steering counts.

## V. CONCLUSION AND LIMITATIONS

In this work, we present RoboPrompt, a system for steering robot policies with sparse, intuitive human input. By leveraging the properties of diffusion and flow-matching, RoboPrompt enables non-expert users to intuitively guide policy behavior and iteratively improve policy performance using data collected from steered rollouts.

However, while our steering method effectively guides the end-effector pose, it does not yet support controlling dexterous end-effectors, particularly multi-fingered hands. Moreover, our Phase I steering module is not limited to specific robot embodiments or environments. Future work could explore training on larger and more diverse datasets to enable zero-shot steering across a broader range of tasks.

## REFERENCES

- [1] P. Intelligence, K. Black, N. Brown, J. Darpanian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, M. Y. Galliker, D. Ghosh, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, D. LeBlanc, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, J. Tanner, Q. Vuong, H. Walke, A. Walling, H. Wang, L. Yu, and U. Zhilinsky, “ $\pi_{0.5}$ : a vision-language-action model with open-world generalization,” 2025. [Online]. Available: <https://arxiv.org/abs/2504.16054>
- [2] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu *et al.*, “Rt-1: Robotics transformer for real-world control at scale,” *arXiv preprint arXiv:2212.06817*, 2022.
- [3] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi *et al.*, “Openvla: An open-source vision-language-action model,” *arXiv preprint arXiv:2406.09246*, 2024.
- [4] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid *et al.*, “Rt-2: Vision-language-action models transfer web knowledge to robotic control,” in *Conference on Robot Learning*. PMLR, 2023, pp. 2165–2183.

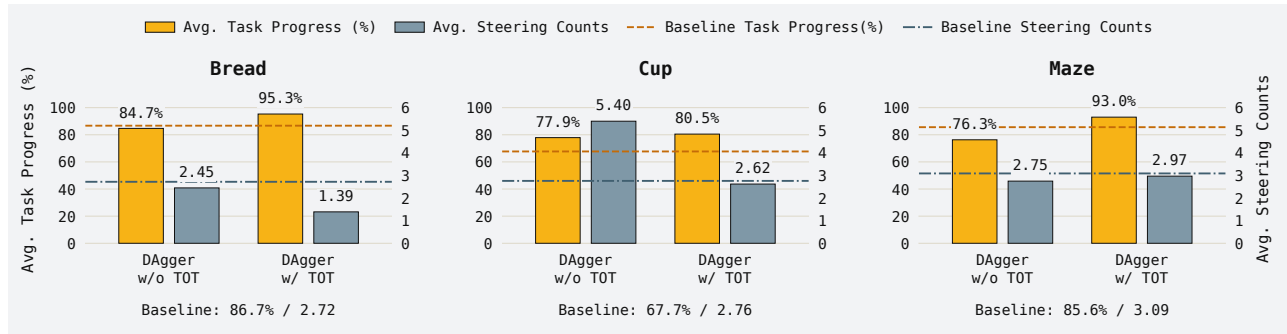


Fig. 10: Policy improvement with filtered (w/ TOT) and unfiltered (w/o TOT) rollout data. Each setting uses 20 successful trajectories for one DAGger round. Baselines show task progress and average steering counts before training.

- [5] T. Yuan, Z. Dong, Y. Liu, and H. Zhao, “Fast-wam: Do world action models need test-time future imagination?” *arXiv preprint arXiv:2603.16666*, 2026.
- [6] S. Ye, Y. Ge, K. Zheng, S. Gao, S. Yu, G. Kurian, S. Indupuru, Y. L. Tan, C. Zhu, J. Xiang *et al.*, “World action models are zero-shot policies,” *arXiv preprint arXiv:2602.15922*, 2026.
- [7] L. Li, Q. Zhang, Y. Luo, S. Yang, R. Wang, F. Han, M. Yu, Z. Gao, N. Xue, X. Zhu *et al.*, “Causal world modeling for robot control,” *arXiv preprint arXiv:2601.21998*, 2026.
- [8] Q. Zhang, L. Li, L. Zhang, S. Yang, Y. Luo, S. Li, R. Wang, J. Wang, J. Shao, G. Xu *et al.*, “Native video-action pretraining for generalizable robot control,” *arXiv preprint arXiv:2607.08639*, 2026.
- [9] K. Goldberg, “Good old-fashioned engineering can close the 100,000-year ‘data gap’ in robotics,” p. eaea7390, 2025.
- [10] A. Liu, B. Feng, B. Wang, B. Wang, B. Liu, C. Zhao, C. Deng, C. Ruan, D. Dai, D. Guo *et al.*, “Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model,” *arXiv preprint arXiv:2405.04434*, 2024.
- [11] K. Team, Y. Bai, Y. Bao, Y. Charles, C. Chen, G. Chen, H. Chen, H. Chen, J. Chen, N. Chen *et al.*, “Kimi k2: Open agentic intelligence,” *arXiv preprint arXiv:2507.20534*, 2025.
- [12] P. Intelligence, B. Ai, A. Amin, R. Aniceto, A. Balakrishna, G. Balke, K. Black, G. Bokinsky, S. Cao, T. Charbonnier *et al.*, “ $\pi_{0.7}$ : A steerable generalist robotic foundation model with emergent capabilities,” *arXiv preprint arXiv:2604.15483*, 2026.
- [13] P. Intelligence, A. Amin, R. Aniceto, A. Balakrishna, K. Black, K. Conley, G. Connors, J. Darpinian, K. Dhabalia, J. DiCarlo *et al.*, “ $\pi_{0.6}^*$ : A vla that learns from experience,” *arXiv preprint arXiv:2511.14759*, 2025.
- [14] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” *The International Journal of Robotics Research*, vol. 44, no. 10-11, pp. 1684–1704, 2025.
- [15] C. Glossop, W. Chen, A. Bhorkar, D. Shah, and S. Levine, “Cast: Counterfactual labels improve instruction following in vision-language-action models,” *IEEE Robotics and Automation Letters*, 2026.
- [16] S. Fei, S. Wang, J. Shi, Z. Dai, J. Cai, P. Qian, L. Ji, X. He, S. Zhang, Z. Fei *et al.*, “In-depth robustness analysis for vision-language-action models,” 2025.
- [17] H. Yu, J. Zhao, Y. Liu, K. Li, C. Ma, D. Zhang, Y. Hu, G. Chen, J. Xie, J. Guo *et al.*, “Point what you mean: Visually grounded instruction policy,” *arXiv preprint arXiv:2512.18933*, 2025.
- [18] H. Huang, M. Cen, K. Tan, X. Quan, G. Huang, and H. Zhang, “Graphcot-vla: A 3d spatial-aware reasoning vision-language-action model for robotic manipulation with ambiguous instructions,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, no. 22, 2026, pp. 18 324–18 332.
- [19] M. Pan, S. Feng, Q. Zhang, X. Li, J. Song, C. Qu, Y. Wang, C. Li, Z. Xiong, Z. Chen *et al.*, “Sop: A scalable online post-training system for vision-language-action models,” *arXiv preprint arXiv:2601.03044*, 2026.
- [20] W. Yu, J. Lv, Z. Ying, Y. Jin, C. Wen, and C. Lu, “Armada: Autonomous online failure detection and human shared control empower scalable real-world deployment and adaptation,” *IEEE Robotics and Automation Letters*, 2026.
- [21] W. Chen, J. S. Bhatia, C. Glossop, N. Mathihalli, R. Doshi, A. Tang, D. Driess, K. Pertsch, and S. Levine, “Steerable vision-language-action policies for embodied reasoning and hierarchical control,” *arXiv preprint arXiv:2602.13193*, 2026.
- [22] R. Zheng, Y. Liang, S. Huang, J. Gao, H. Daumé III, A. Kolobov, F. Huang, and J. Yang, “Tracevla: Visual trace prompting enhances spatial-temporal awareness for generalist robotic policies,” in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 54 277–54 296.
- [23] T. Lin, Y. Zhong, Y. Du, J. Zhang, J. Liu, Y. Chen, E. Gu, Z. Liu, H. Cai, Y. Zou *et al.*, “Evo-1: Lightweight vision-language-action model with preserved semantic alignment,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2026, pp. 13 397–13 406.
- [24] Z. Wang, Y. Chen, Y. Liu, J. Ye, P. Chen, C. Lu, S. Liu, B. Yu, and J. Jia, “Vp-vla: Visual prompting as an interface for vision-language-action models,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.22003>
- [25] Y. Wang, L. Wang, Y. Du, B. Sundaralingam, X. Yang, Y.-W. Chao, C. Pérez-D’Arpino, D. Fox, and J. Shah, “Inference-time policy steering through human interactions,” in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 15 626–15 633.
- [26] M. Kelly, C. Sidrane, K. Driggs-Campbell, and M. J. Kochenderfer, “Hg-dagger: Interactive imitation learning with human experts,” in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 8077–8083.
- [27] H. Liu, S. Nasiriany, L. Zhang, Z. Bao, and Y. Zhu, “Robot learning on the job: Human-in-the-loop autonomy and learning during deployment,” *The International Journal of Robotics Research*, vol. 44, no. 10-11, pp. 1727–1742, 2025.
- [28] P. Wu, Y. Shentu, Q. Liao, D. Jin, M. Guo, K. Sreenath, X. Lin, and P. Abbeel, “Robocopilot: Human-in-the-loop interactive imitation learning for robot manipulation,” *arXiv preprint arXiv:2503.07771*, 2025.
- [29] S. Luo, Q. Peng, J. Lv, K. Hong, K. R. Driggs-Campbell, C. Lu, and Y.-L. Li, “Human-agent joint learning for efficient robot manipulation skill acquisition,” in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 1370–1377.
- [30] A. Tang, W. Chen, A. Wagenmaker, C. Finn, and S. Levine, “Improving robotic generalist policies via flow reversal steering,” *arXiv preprint arXiv:2606.13675*, 2026.
- [31] S. Ross, G. Gordon, and D. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Proceedings of the fourteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings, 2011, pp. 627–635.
- [32] J. Fang, W. Chen, H. Xue, F. Zhou, T. Le, Y. Wang, Y. Zhang, J. Lv, C. Wen, and C. Lu, “Robopocket: Improve robot policies instantly with your phone,” *arXiv preprint arXiv:2603.05504*, 2026.